To build these endpoints according to the system architecture, you will need to focus on **monitoring infrastructure** and **detailed AI call debugging**.

Here is how you can implement them based on the documentation:

## 1. GET /health (System Health Status)

This endpoint is used for automated smoke tests and monitoring the health of the backend and AI pipelines. It should return more than just a "200 OK"—it needs to reflect the operational status of your services.

- **Error Tracking:** It should report if any core services (Cloud SQL, Cloud Storage) are unreachable.
- **AI API Health (Latency):** It should track the "fine-tune and panel detection health". This includes monitoring the success rate of Vertex AI (Gemini) and Document AI (OCR) calls.
- **Active Usage:** While not a "live user count," it can report stats like the number of reports currently in a "processing" status.
- **Response Structure:** It should return a JSON object with statuses for database, storage, vertex_ai, and ocr_service.

## 2. GET /internal/traces/{message_id} (LLM Traces)

This is an **Admin-only** debugging endpoint designed to see exactly what happened during a specific AI interaction.

- **How it works:** When an AI response is generated, a row is written to the LLM_TRACES table . This endpoint retrieves that specific row using the message_id.
- **Key Data to Return:**
  - **The Prompt (assembled_prompt_enc):** This is the most important field for debugging. It contains all four "Zones" of the prompt (System, Context, History, and User Question) concatenated together .
  - **The Raw Response (raw_response_enc):** The exact text Gemini returned *before* any safety guardrails or formatting were applied.
  - **Timing Breakdown:** Return llm_duration_ms (the Gemini call itself), retrieval_duration_ms (vector search), and total_duration_ms .
  - **Guardrail Outcomes:** Show if guardrail_fired is true and which specific check (e.g., cardiac_urgency_override or triage_diagnosis_pattern) triggered it .

- **Retrieval Metadata:** For document chats, return the `cosine_scores` and `retrieved_chunk_ids` to see which parts of the medical record the AI "read" .

## Implementation Details

- **Encryption:** Remember that both the prompt and the raw response are stored using **AES-256 encryption** in the database because they contain sensitive patient information (PHI) . Your internal endpoint must decrypt this data before showing it to the admin.

- **Retention:** These traces are only kept for **90 days** for operational debugging. After 90 days, the encrypted prompt and response columns are automatically cleared by a nightly pipeline to reduce PHI risk.

- **Security:** This endpoint must be protected by an admin-level role check (`role = 'admin'`) and should never be accessible to standard patients.

Based on the project documentation, the **POST /internal/ft-pipeline/run** and **GET /internal/ft-pipeline/stats** endpoints exist alongside the automated nightly pipeline for three specific reasons:

## 1. Manual Override & On-Demand Training

While the pipeline is designed to run automatically at night (e.g., at 4:00 AM) to prepare fine-tuning data, an admin may need to trigger it manually to:

- **Force an Immediate Update:** If a new model version is being prepared and you need the latest data immediately rather than waiting for the next nightly cycle.

- **Reprocess After a Failure:** If the automated nightly job fails (e.g., due to a database connection issue), an admin can use this endpoint to restart the process without waiting 24 hours.

## 2. Monitoring & Debugging (/stats)

The automated job runs in the background, but admins need a way to see what it is doing. The /stats endpoint allows them to:

- **Check Eligibility:** See how many chat messages were marked as ft_eligible by the nightly logic .

- **Verify Quality:** Monitor the "Fine-tune pipeline health" and see why certain data was excluded (using the ft_excluded_reason field).

## 3. Safety & Review Gate

The fine-tuning process is highly sensitive because it involves medical data. The documentation notes that:

- **PHI Scrubbing:** Data only enters the FINE_TUNING_EXAMPLES table after passing a PHI scan and verifying **Research Consent** from the user.

- **Clinician Review:** In some cases, a clinician must review the generated examples. Having an admin endpoint allows a human to control when a batch is finalized and exported for model training.

## Summary of Differences

| Feature | Automated Nightly Job | Admin Endpoints (/internal) |
|---|---|---|

| | | |
|---|---|---|
| **Trigger** | Scheduled (e.g., Cron/Cloud Scheduler) | Manual (Admins only) |
| **Primary Goal** | Routine maintenance and data preparation | Debugging, stats, and emergency reprocessing |
| **Data Scope** | Processes all new data from the last 24 hours | Can be used to trigger specific batches or view totals |

In short, the automation handles the **routine**, while the internal endpoints provide the **control and visibility** needed for system maintenance.

As per the project documentation, the **nightly pipeline**—which handles data cleanup and fine-tuning preparation—is intended to run as an automated background process.

## 1. Automation Method

The documentation specifies that these tasks are handled by a **nightly background job**. While the document doesn't explicitly name "Cron" in every section, it describes the mechanism as a **"Background Task Queue"** or a **"nightly job"** that runs independently of user actions.

- **Cleanup Tasks:** A background job automatically deletes expired unclaimed guest rows after 24 hours and removes user reports once they pass their 2-year retention limit.
- **Trace Maintenance:** A nightly job is specifically required to "nullify" (clear) the encrypted prompt and response columns in the LLM_TRACES table for records older than 90 days.
- **Fine-Tuning:** The calculation of which chat messages are ft_eligible (eligible for fine-tuning) is also computed automatically on a nightly basis.

## 2. Implementation Choice

For your current backend development (using FastAPI on Google Cloud Run), you have two primary ways to "define" and trigger this:

- **Option A: Cloud Scheduler (Recommended):** This is the Google Cloud equivalent of a **Cron job**. You can define a private endpoint in your backend (e.g., POST /internal/ft-pipeline/run) and configure Cloud Scheduler to trigger it automatically every night at a specific time.
- **Option B: Background Tasks:** While FastAPI has a BackgroundTasks feature, it is typically used for immediate tasks (like starting OCR right after an upload) rather than scheduled nightly runs .

## 3. Summary of Choice

You **do not** have to define a manual "feature" for a user to trigger this; it must be automatic to satisfy data privacy and retention requirements (like PDPL and HIPAA-adjacent standards). Your role is to build the service logic (the "what" to clean) and then wire it to a scheduler (the "trigger").

Exactly, you've understood the core "AI Nurse" concept perfectly. Based on the system architecture, here is the breakdown of how that relationship works between reports, sessions, and symptoms:

## 1. One "AI Nurse" per Report (The Document Session)

The system is built around a **one-to-one relationship** between a chat session and a specific medical report.

- **Initialization:** When you start a POST /chat/sessions with mode="document", you must provide a report_id.
- **Isolated Knowledge:** The "AI Nurse" for that session only "sees" the data from that specific report. It uses **RAG (Retrieval-Augmented Generation)** to search only the embeddings tied to that report_id.
- **Continuous Symptoms:** You can mention many different symptoms over multiple messages (e.g., "I feel dizzy," then later "I also have a headache"), and the AI will keep checking those symptoms against the data in that **single** report.

## 2. Can you attach a second document to the same chat?

**No, not in the current MVP architecture.** * **Report-Centric:** Each session is scoped to one report_id in the CHAT_SESSIONS table.

- **Switching Reports:** If a user wants to discuss a different report (e.g., moving from a blood test to an MRI finding), they would start a **new session** for that specific report.
- **Exception (Multi-Panel Reports):** If a user uploads a single "Full Comprehensive Checkup" that contains many panels (like CBC, Lipid, and Thyroid all in one PDF), the AI *can* see all of them because they are all part of the same report_id.

## 3. The "Three Stages" of the AI Nurse

The AI adapts its behavior based on what information it has:

| Stage | Mode | AI Logic |
|---|---|---|

| | | |
|---|---|---|
| **1. Only Symptoms** | triage | Acts as a **Symptom Triage** bot. It doesn't look at any reports and just asks structured questions to give a Green/Yellow/Red safety result. |
| **2. Only Document** | document | Acts as a **Document Summarizer**. It explains the lab values and flags what is abnormal without asking the user about symptoms. |
| **3. Doc + Symptoms** | document | Acts as **Document-Based Triage**. This is the most "advanced" stage where the AI explains your report *and* uses your symptoms to give a severity classification. |

## 4. How the AI Knows a Document is Related

This is handled purely through the **Session ID** and **Report ID** link in your database:

- When a user sends a message, the backend looks up the session_id.
- It finds the report_id attached to that session.
- It fetches the **Top-5 text chunks** from that report's embeddings that are most similar to the user's message.
- It sends those chunks to Gemini as the "Ground Truth," effectively telling the AI: *"Answer the user's symptoms using only this specific medical report context"*.

## 5. Multi-Check Urgency

The "AI Nurse" is constantly monitoring for danger. If a document has a cardiac_urgency_flag (e.g., high Troponin), the AI is **forced** to show a red warning banner in the chat, regardless of

what symptoms the user mentions.

Based on the system architecture documentation, here is how the AI manages multiple documents, relates them to your messages, and handles medical urgency across a chat session.

## 1. How the AI relates Documents to Messages

The AI does not "guess" which document you are talking about; it uses a structured **Retrieval-Augmented Generation (RAG)** pipeline.

- **Session Binding:** When you start a chat session, it is linked to a specific report_id in the database.

- **Vector Search (RAG):** When you send a message like "What does this value mean?", the system takes your text, converts it into a mathematical vector (embedding), and searches the EMBEDDINGS table *only* for the chunks belonging to that specific report_id.

- **Context Injection:** The top-5 most relevant text chunks from your report are then "injected" into the hidden prompt sent to the AI (Gemini). The AI then answers your symptoms based *only* on those specific report pieces.

## 2. Attaching Multiple Documents in One Session

The current MVP architecture is designed for **single-report focus** per session, but it handles "Multi-Panel" documents extensively.

- **Single Session, Single Report:** Currently, a chat session is initialized with one report_id. If you want to talk about a completely different report (e.g., switching from a Blood Report to an MRI text finding), you would typically start a new session.

- **Multi-Panel Reports:** If you upload a "Full Checkup" that contains many panels (CBC, Lipid, Liver, etc.), the system treats this as one report but detects all the different panels.

- **Round-Robin Retrieval:** In these cases, the AI uses a "round-robin" search. If you ask a broad question, it pulls information from across all the different panels in that one document to ensure it doesn't miss anything.

## 3. How Urgency is Checked (The "Three Checks")

The system does not wait for you to ask a question to check for danger. It uses a **multi-layered safety architecture**:

1. **Layer 1: The "At-Upload" Check (Background):** As soon as you upload, the backend scans for "cardiac enzymes" (like Troponin). If it finds high values, it sets a permanent cardiac_urgency_flag = TRUE on that report.

2. **Layer 2: The "Session-Start" Check:** If you open a chat for a report with that flag, the AI is **forced** to prepend a prominent medical warning to its first response, regardless of what you asked.

3. **Layer 3: The "Mid-Chat" Guardrails:** Every single time the AI generates a response, a "Layer 2 Guardrail" scans the text. If the AI tries to downplay a "Red" urgency level or misses a mandatory disclaimer, the system blocks that response and replaces it with a safe, pre-written fallback message.

## Summary of Urgency Scoring

| Result Level | Urgency Score | Action |
|---|---|---|
| **Red** | 7–10 | High-priority alert; emergency recommendation; triggered by specific patterns like chest pain + shortness of breath. |
| **Yellow** | 4–6 | "Worth checking out"; suggests a doctor visit within a few days. |
| **Green** | 1–3 | "All good for now"; standard health monitoring. |

You have the core structure correct, though the documentation clarifies some specific limits and behaviors for each mode.

Here is the breakdown of the three conversation levels based on the system architecture:

## 1. Symptom Triage (mode="triage")

- **Target User:** Available for **Guests** (unauthenticated, first session only) or **Registered Users**.
- **Message Limit:** The documentation does not specify a hard 20-message limit, but it requires a **minimum of 4 patient turns** before the "Get My Results" option becomes available.
- **AI Behavior:** The AI acts as a "Symptom Triage" bot, asking one structured question at a time to determine health severity.
- **Output:** Generates a **Triage Result** screen with a **Green, Yellow, or Red** severity classification, an urgency score (1–10), a summary of symptoms, and a medical recommendation .

## 2. Document Summary (mode="document")

- **Trigger:** Activated when a user uploads a health document (PDF or image).
- **Message Limit:** This is a **single-turn response**. It is not a multi-turn chat by default; rather, it's a generated summary.
- **AI Behavior:** The AI generates a structured summary of the lab panels (e.g., CBC, Lipid). It flags abnormal values and provides plain-language explanations.
- **Urgency:** If the report contains high cardiac markers (e.g., Troponin), it **automatically sets a cardiac urgency flag** and prepends a mandatory warning to the summary.

## 3. Document-Based Triage (mode="document")

- **Trigger:** A user engages in a chat session referencing an uploaded document.
- **Message Limit:** While the rate limit is set at **10 chat messages per minute**, the system uses a 30-day idle timeout for sessions, and the technical specifications don't cite a strict 20-message cap per session.

- **AI Behavior:** This is the "AI Nurse" mode. It uses **Retrieval-Augmented Generation (RAG)** to answer symptom questions by looking *specifically* at the data in the attached report.

- **Output:** Provides a **combined output**—it explains the document findings and generates a final **Triage Result** (urgency score and recommendation) based on the full conversation.

Key Rate Limits to Note:

- **General Chat:** 10 messages per minute per user.
- **API Requests:** 60 requests per minute per user.
- **Uploads:** 5 document uploads per day per user.
- **Guest Triage:** Limited to 1 session per device/IP every 24 hours.

The documentation defines **three distinct conversation modes** (levels) for creating a session. These modes are selected when calling the POST /chat/sessions endpoint using the mode parameter.

## 1. Symptom Triage (mode="triage")

- **Trigger:** The patient describes symptoms; **no document is required**.
- **User Status:** Can be used by a **Guest** (unauthenticated) for the first session or a logged-in user.
- **AI Logic:** The system asks a series of structured symptom questions (minimum 4 turns) to determine severity.
- **Output:** Generates a **Triage Result** screen with a Green, Yellow, or Red severity classification and an urgency score.

## 2. Document Summary (mode="document")

- **Trigger:** The patient uploads a health document.
- **Requirement:** Must provide a report_id for a report that has reached "ready" status.
- **AI Logic:** A single-turn summary is generated based on the specific lab panels detected (e.g., CBC, Lipid). No triage questions about symptoms are asked in this mode.
- **Output:** An **AI summary screen** showing structured findings, abnormal value flags, and plain-language explanations.

## 3. Document-Based Triage (mode="document")

- **Trigger:** The patient uploads a document **AND** engages in a symptom discussion that references that document.
- **AI Logic:** This uses a **Retrieval-Augmented Generation (RAG)** pipeline. The AI answers questions about symptoms by looking at the specific data inside the attached medical report.
- **Output:** A **combined output** consisting of a summary of the document findings plus a Triage Result (severity score and recommendation) based on the symptoms described.

Comparison of Session Types

| Mode | Document Needed? | Symptom Questions? | Primary Goal |
|---|---|---|---|
| **Symptom Triage** | No | Yes | Get a severity/urgency score (Green/Yellow/Red). |
| **Document Summary** | Yes | No | Understand what is in a lab report. |
| **Document-Based Triage** | Yes | Yes | Combine report data with symptoms for a safe recommendation. |